The Control Plane: Routing at Scale

Link-State, The Adjacency Matrix, and OSPF

Bernal Manzanilla

Concordia University of Edmonton
IT 213: Networking

Winter 2026

Agenda

- **Part 1: Tying the Planes Together**
  - Control Plane logic to Link-Layer Frames
- **Part 2: Link-State Routing & Graph Theory (Ch. 5.2)**
  - The Adjacency Matrix (Cost Matrix)
  - Dijkstra's Algorithm Pseudocode
  - The $\mathcal{O}(n^2)$ Scalability Limit
- **Part 3: Scaling the Internet (Ch. 5.3 & 5.4)**
  - Autonomous Systems (AS)
  - Intra-AS Routing: OSPF
  - Inter-AS Routing: BGP

Bridging the Control Plane and the Link Layer

Before diving into routing algorithms, let's recall how the Control Plane interacts with the Data Link Layer we studied last week:

- **The Brains (Control Plane):** Routing algorithms compute the end-to-end path from source to destination. This logic populates the router's forwarding table.
- **The Brawn (Data Plane):** When a packet arrives, the router looks up the destination IP in the forwarding table to find the correct output port.
- **The Delivery (Link Layer):** The router cannot just push raw IP datagrams onto a wire. It must encapsulate the datagram into a **Link-Layer Frame**, attaching MAC addresses to physically move the data to the next adjacent router.

Link-State (LS) Routing Algorithms (Section 5.2)

In a Link-State algorithm, the network topology and all link costs are known to every router.

- **Global Knowledge:** Every router broadcasts its link-state information to *all* other routers in the autonomous system.
- **Complete Map:** As a result, all nodes build an identical, complete mathematical map of the network in their memory.
- **Graph Theory:** We model the network as a graph $G = (N, E)$, where N is the set of routers (nodes) and E is the set of physical links (edges) connecting them.

The Adjacency Matrix: The Link-State Database

How does a router actually store this graph in memory? It builds an **Adjacency Matrix** (specifically, a Cost Matrix).

- Let $c(x, y)$ be the cost of the physical link between router x and router y.
- If nodes x and y are directly attached, $c(x, y)$ is a positive value.
- If nodes x and y are NOT directly attached, the cost is infinity: $c(x, y) = \infty$.

For a tiny 3-node network (A, B, C), the router holds a 2D array that looks exactly like this:

$$\begin{bmatrix} 0 & c(A,B) & \infty \\ c(B,A) & 0 & c(B,C) \\ \infty & c(C,B) & 0 \end{bmatrix}$$

Dijkstra's Algorithm: Pseudocode (Section 5.2)

Once the matrix is built, the router executes this algorithm to find the shortest paths:

```
Initialization:
    N' = {u}
    for all nodes v
        if v is a neighbor of u
            then D(v) = c(u,v)
        else D(v) =
Loop
    find w not in N' such that D(w) is a minimum
    add w to N'
    update D(v) for each neighbor v of w and not in N':
        D(v) = min( D(v), D(w) + c(w,v) )
    /* new cost to v is either old cost to v or known
       shortest path cost to w plus cost from w to v */
until N' = N
```

The Scalability Limit of Link-State (Section 5.2)

Dijkstra's algorithm works flawlessly on a small scale, but it collapses when applied to the entire Internet.

- **Complexity:** For a network of n nodes, the algorithm must search through all nodes not currently in the shortest-path tree.
- **Access Times:** Provided no evident sort or hashing method exists to access all the link states in constant time, table access times grow at best at $\mathcal{O}(n)$ access complexity, leading to an overall algorithmic complexity of $\mathcal{O}(n^2)$.
- **Broadcast Overhead:** With hundreds of millions of routers on the Internet, the overhead of broadcasting link-state updates to every single router would consume all available bandwidth.
- **Conclusion:** This access complexity explicitly precludes the deployment of Link-State algorithms at a global Internet scale.

The Solution: Autonomous Systems (Section 5.3)

To solve the $\mathcal{O}(n^2)$ scalability problem, the Internet is partitioned into **Autonomous Systems (ASs)**.

- An Autonomous System is a group of routers that are typically under the same administrative control (e.g., a university network, an ISP, or a corporate network).
- Routers within the same AS all run the same routing algorithm.
- By grouping routers into ASs, we limit the scope of Link-State broadcasts. A router only needs to build a matrix of its *own* AS, heavily reducing n.

Intra-AS Routing: OSPF (Section 5.3)

Open Shortest Path First (OSPF) is the most widely deployed intra-AS routing protocol in the Internet.

- **Algorithm:** OSPF is a Link-State protocol. It uses the Adjacency Matrix and Dijkstra's algorithm.
- **Broadcasting:** A router broadcasts routing information to all other routers *in the autonomous system*, not the whole Internet.
- **Advanced Features:** OSPF includes security (all messages are authenticated to prevent malicious router injections) and multiple same-cost paths (load balancing).

Inter-AS Routing: BGP (Section 5.4)

If OSPF handles routing *inside* the Autonomous System, how do packets travel *between* different Autonomous Systems?

- **Border Gateway Protocol (BGP):** BGP is the de facto standard inter-AS routing protocol. It is the "glue" that holds the Internet together.
- **Path Vector:** BGP is not a Link-State protocol. It does not build an adjacency matrix. Instead, BGP routers advertise entire paths (the sequence of ASs) to reach a specific subnet prefix.
- **Policy over Performance:** While OSPF routes based on the shortest path in the matrix, BGP routes based on **policy**. An ISP might configure BGP to never route traffic through a competitor's network.